1. NLP Basics & Challenges

Goal: interdisciplinary field (CS + AI + ling.); enable computers to understand human instructions and perform tasks. LLM paradigm shift: *one big model* replaces task-specific models.

Types of Ambiguity (map example→type)

- **Word Sense:** one word, many meanings. "I saw her *duck*" (animal / verb).
- **PP Attachment:** modifier attachment. "I saw her duck *with a telescope*" (who has telescope).
- **Headline/Syntactic:** compressed phrasing. "San Jose cops kill man with knife"; "Local High School Dropouts Cut in Half"; "Include your children when baking cookies".
- **Pronoun Ref:** unclear referent. "Umbrella can't fit in bin because *it* is too big/small"; "doctor brings dog to patients, *he* loves sloppy kisses".

Modern Challenges

- **Scalability:** Penn Treebank ≈1M → Wikipedia ≈2.9B → Web (billions).
- **Dynamic Language:** slang evolves (LOL, G2G, BFN, FWIW).
- **Hallucination:** plausible but false (fake bios, miscount baseballs 20→24).

Tasks (map problem→task)

MT, Sentiment, Info Extraction (unstructured→DB), QA (Watson/Siri), Math solvers (AlphaGeometry), agents, embodied robots.

2. Coreference & Evaluation

Coref: group mentions (pronouns, common nouns, NEs) referring to same entity.

Mention Edge Cases

- *Not all pronouns are mentions*: "**It** is raining" → pleonastic, no referent.
- *Nested*: "[[my] values]". *Boundary*: "US President Biden" vs. "US President Biden, who is in Europe".

MUC (Link-based)

P/R on mention **pairs**.

$$P = \frac{TP\ pairs}{\text{pred pairs}},\ R = \frac{TP\ pairs}{\text{gold pairs}}$$

Trap: One giant cluster ⇒ $R = 100\%$ (all gold pairs are inside). Exact-match is the opposite extreme—too harsh.

B-CUBED (Mention-based)

P/R *per mention*, then average. Fixes MUC giant-cluster loophole.

F1 Aggregation

$$F1 = \frac{2PR}{P+R}$$

- **Macro F1:** F1 per doc, then mean.
- **Micro F1:** pool TP/FP/FN across all docs → one global F1.

3. Word Reps & PMI

One-Hot (key flaws)

Dim = $|V|$ (Google 1T: ∼13M); sparse; OOV→no vector; **dot product of any two different one-hots** $= 0$ ⇒ **cannot capture similarity**. $v_{happy} \cdot v_{sad} = 0$.

Taxonomy (WordNet)

Manual synonym/hypernym hierarchy. Labor, subjective, misses nuance ("proficient"≠"good" always), can't track slang ("badass", "nifty").

Distributional Hypothesis

"A word is characterized by the company it keeps." Build co-occurrence matrix; SVD/PCA → low-rank $d \times k$ compressed reps.

PMI (expect a word-problem)

$$PMI(x,y) = \log_2 \frac{P(x,y)}{P(x)P(y)} \in (-\infty, +\infty)$$

PPMI $= \max(PMI, 0)$ (negatives often noisy).

Compute from matrix (step-by-step):

1. N = sum of all cells.
2. $P(x) = \frac{\text{row sum of } x}{N}$, $P(y) = \frac{\text{col sum of } y}{N}$.
3. $P(x,y) = \frac{\text{cell}(x,y)}{N}$.
4. Plug into formula. Use $\log_2$ (not ln!).

4. Word2Vec (Skip-Gram)

Given center word, predict context words in a window.

$$P(o|c) = \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)}$$

Loss: $J = -\sum_t \sum_{j \neq 0} \log P(w_{t+j}|w_t)$.

Parameters (classic Q)

Every word has TWO vectors: center v_w *and* context u_w ⇒ total params = $\mathbf{2|V|d}$.

Trap: Answering $|V|d$ forgets the context vectors.

Gradient Intuition

$$\frac{\partial J}{\partial v_c} = u_o - \mathbb{E}_{w \sim P(w|c)}[u_w]$$

"Observed outside vector minus expected (probability-weighted) context vector."

Negative Sampling

Softmax over entire $|V|$ is too expensive → sample a few random "negative" non-context words; cheap approximation used in training.

Similarity & Analogies

Cosine $= \frac{u \cdot v}{\|u\|\|v\|}$. *If vectors are unit-norm, dot product = cosine.*

Analogy: $v_{king} - v_{man} + v_{woman} \approx v_{queen}$.

Polysemy

"bank" = river/finance. W2V stores senses as a **linear superposition** in a single vector; sparse-coding can separate them post-training.

Bias

Training data biases leak in: $v_{surgeon} - v_{man} + v_{woman} \approx v_{nurse}$.

5. N-Gram LMs

Chain Rule

$$P(w_1..w_T) = \prod_{t=1}^{T} P(w_t|w_{<t})$$

Markov / MLE

Bigram (depend on 1 prev), Trigram (2 prev).

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Ex: "I am" appears 3x, "I" 3x ⇒ $P(\text{am}|\text{I}) = 1$.

Zipf & Sparsity

r-th most common word: freq $\propto 1/r$. Two failure modes:

- Numerator 0 ⇒ whole sentence prob = 0.
- Denominator 0 ⇒ undefined.

Add-One Laplace

$$P = \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + V}$$

Trap: Must add $+\mathbf{V}$ (vocab size) to denominator, NOT +1, so probs sum to 1.

Add-λ

$$P = \frac{C + \lambda}{C(w_{i-1}) + \lambda V}$$

λ too large (e.g., 1000) ⇒ high bias (over-smooth); too small (0.001) ⇒ high variance. Pick on dev set.

Eval: Perplexity

$PP = P(w_1..w_T)^{-1/T}$. Lower better.

6. Neural LMs

Feedforward (Fixed Window)

Quiz 1 favorite. Concatenate last $n-1$ embeddings → MLP → softmax.

+ Dense ⇒ no sparsity; captures similarity.
– **Fixed** window; can't take variable lengths; no cross-position param sharing.

RNN LM

Same W_h, W_e at *every* step (**symmetry / weight sharing**).

$$h_t = \sigma(W_h h_{t-1} + W_e x_t + b),\ \hat{y}_t = \text{softmax}(Uh_t)$$

+ Variable length; shared params; context grows.
– Sequential (slow); **vanishing gradient** from repeated multiplication by values $\in (-1, 1)$ during BPTT ⇒ can't learn long-range.

LSTM Gates

Cell state c_t + gates: **forget** f_t (drop), **input** i_t (add), **output** o_t (expose).

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,\ h_t = o_t \odot \tanh(c_t)$$

Gates mitigate vanishing gradient.

7. Seq2Seq & Decoding

Seq2Seq Bottleneck

Encoder RNN → *single* fixed vector → Decoder. Info loss on long sentences.

Greedy

arg max per step. Fast, no undo, globally suboptimal.

Beam Search (k)

Keep top-k partial hypotheses by cumulative log-prob.

Trap: $k = 1$ ⇒ beam ≡ greedy. Larger k: more search, but very large k can hurt fluency.

Temperature T

Softmax $\text{softmax}(z/T)$.

- $T \to \infty$: flatter, more random/diverse.
- $T \to 0$: sharper, becomes **greedy** (argmax).

Sampling

Top-k: sample from top k tokens. **Nucleus (Top-p):** smallest set with cumulative prob $\geq p$ (e.g., 0.8).

8. Attention

Decoder looks at *every* encoder hidden state each step.

Mechanism (step t)
1. Score $e_{t,i} = s_t^\top h_i$ (dot) or additive MLP.
2. $\alpha_{t,i} = \text{softmax}(e_{t,i}) \Rightarrow$ **weights** $\in [0,1]$, $\sum_i \alpha_{t,i} = 1$.
3. Context $c_t = \sum_i \alpha_{t,i} h_i$.
4. Combine with s_t, predict w_t.

4 Benefits ("which is NOT true?")
(i) **Performance** ↑; (ii) **Solves bottleneck**; (iii) **Helps vanishing gradient** (shortcut path); (iv) **Interpretability** via soft alignment viz.

Trap: Attention weights are the **softmaxed** α's, not the raw scores. They sum to 1, NOT to the number of words.

9. Transformers / Self-Attn

No recurrence ⇒ parallel over sequence.

Trap: Transformers do NOT process tokens sequentially (Quiz 2 Q1).

Attention looks cross (dec→enc). **Self-**attention looks within *same* sentence.

Q, K, V
$Q = XW^Q$, $K = XW^K$, $V = XW^V$. Tokens *query* others (via $Q \cdot K$), pull info from *values* V.

$$\text{Attn} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Space Complexity (Quiz 2 Q4)
$QK^\top : (L \times D)(D \times L) = L \times L$ matrix.

$$\boxed{\text{Space/Time} = O(L^2)}$$

Scales **quadratically** in sequence length L.

Multi-Head
h parallel (Q, K, V) heads capture different relations (pronouns, verbs, syntax).

Masked Self-Attn (Decoder)
Causal: future positions set to $-\infty$ *before* softmax ⇒ probability 0 after. Prevents peeking ahead.

Positional Encodings
Needed—self-attention alone is permutation-invariant.

10. Implementation (HF + GPU)

GCP / VM
Default GPU quota = 0; request takes 24–48h. Pick "**Deep Learning on Linux**" boot disk (CUDA pre-installed). **Stop** VM when unused. Switching billing accounts: no downtime.

Linux / GPU Commands
- `nvidia-smi` – GPU status, mem, processes.
- `export CUDA_VISIBLE_DEVICES="0"` – pin GPU 0. `=""` forces CPU.
- `tmux new -s exp1` – new session.
- `Ctrl+b`, then `d` – detach.
- `tmux a -t exp1` – reattach. `tmux ls` – list.

HF AutoModel
`AutoModel.from_pretrained("bert-base-cased")` picks arch from name. Task variants: `AutoModelForSequenceClassification`, `...ForQuestionAnswering`, `...ForMaskedLM`.

HF AutoTokenizer (classic exam)
- `.tokenize(s)` → list of subword strings (e.g., `['i','like','nl','##p']`).
- `.convert_tokens_to_ids(t)` → integer IDs.
- `.encode(s)` → **tokenize + convert + add special tokens**. BERT: prepends `[CLS]=101`, appends `[SEP]=102`.
- `.decode(ids)` → string (keeps special tokens).

Trap: `encode` for BERT adds 2 extra IDs (101, 102) beyond the words.

Batch: auto **padding** (0s) + **truncation**; outputs `attention_mask` marking real vs. pad.

Word-Problem Targeting

- *"Model groups all mentions into 1 cluster"* → MUC recall = 100%; use B-CUBED.
- *"Given co-occurrence matrix, compute PMI"* → normalize by N, get marginals, $\log_2$ ratio.
- *"# params in Word2Vec"* → $2|V|d$ (not $|V|d$).
- *"P(word|prev)=0"* → apply Laplace: $+1$ numerator, $+V$ denominator.
- *"Feedforward LM limitation"* → fixed context window, no variable-length input.
- *"Beam size $k = 1$"* → equivalent to greedy.
- *"Attention weights sum to ?"* → 1 (post-softmax), range $[0, 1]$.
- *"Transformer space complexity"* → $O(L^2)$ from $QK^\top$ being $L \times L$.
- *"High temperature effect"* → flatter, more random.
- *"BERT encode output for N words"* → $N + 2$ IDs (CLS + SEP).
- *"Which ambiguity: 'I saw her duck'"* → word-sense (duck noun/verb). With *telescope* → PP attachment.